A Zero-Knowledge Proof of Possession of a Machine-Checked Proof Project

Micro-Lean4 / `meta_lean_export`

(all statements of this paper are Lean 4 theorems in `RequestProject/MicroLeanZKPoP.lean`)

Abstract

The Micro-Lean4 development is a machine-checked polyglot FFI meta-system: a micro-core type language, Rust/C++/JS/Python/emoji emitters, a reference-counting discipline with linear-ownership and mutable-borrow theorems, an invertible canonical interlingua, and — in §12 — an object-level proof system whose proofs (*certificates*) can be serialized, shipped across the FFI boundary and re-checked by a foreign runtime. Transporting a certificate, however, discloses it in full.

This paper adds the complementary capability and proves it correct inside Lean: a three-move Σ-protocol by which a prover publishes a single group element — the *digest* of a certificate — and then convinces any verifier that it really possesses a certificate that the §12 checker accepts for a stated claim, *without revealing a single token of that certificate*. We prove, as Lean theorems with no `sorry` and only the standard axioms: an injective arithmetization of certificates with a computable inverse; completeness; special soundness, upgraded to an *extractor that outputs the certificate itself* and hence, through the §12 soundness theorem, the truth of the claim; a counting bound showing a prover without the discrete logarithm is accepted on at most one of the q challenges, i.e. with probability at most $1/q$; and perfect honest-verifier zero knowledge in the form of an explicit bijection between honest and simulated transcripts, together with the equality of the two transcript sets. A Fiat–Shamir variant makes the argument a single publishable string, and a concrete instantiation discharges every hypothesis by computation, so nothing in the development is vacuous.

Contents

1 Introduction

1.1 What is being proved, and to whom

The claim we want to make in public is of the form

"we hold a machine-checkable proof of c"

where c is a statement about the Micro-Lean4 FFI system — for example *"`Array Nat` is boxed, and the Rust emitter renders it as `Vec<*mut lean_object>`"*. There are two established ways to support such a claim, and both are unsatisfactory on their own:

- **Show the proof.** This is what §12 of the development already supports: a certificate is a finite syntax tree of inference-rule applications, it serializes to a printable ASCII string, and a foreign runtime that receives only that string can parse it and re-run the total checker `check`. Theorem `check_sound` guarantees that whatever the checker accepts is true. But the verifier now *has* the proof.
- **Be believed.** The prover asserts possession. Nothing is verified.

A zero-knowledge proof of possession (ZKPoP) is the third option: the verifier ends up as convinced as in the first case while learning exactly as much as in the second. That is the object constructed and verified here.

1.2 Contributions

Everything below is a Lean 4 theorem in `RequestProject/MicroLeanZKPoP.lean` (namespace `MicroLean.ZKPoP`), building on `RequestProject/MicroLeanProofOfProof.lean` (§12, namespace `MicroLean.PoP`). The project builds with `lake build`, contains no `sorry`, and every theorem cited here depends only on `propext`, `Classical.choice` and `Quot.sound`.

1. **Arithmetization** (§2.2). `certCode` maps a certificate to a natural number (base-32 numeral over the §12 token stream with a nonzero sentinel digit); `certOfCode` inverts it. Round trip: `certOfCode_certCode`; injectivity: `certCode_injective`.
2. **The protocol** (§4). `prove`, `accepts`, `simulate`, `extract` in a cyclic group of prime order q generated by g, for the public digest $y = g^{\texttt{certCode}\, p_0}$.

3. **Completeness** (`completeness`).

4. **Knowledge soundness** (§6): `special_soundness` extracts the discrete logarithm from two accepting transcripts, and `zkpop_extracts_certificate` turns that exponent back into the certificate p_0 itself and concludes `Claim.holds c`.

5. **Soundness error** (`cheating_challenge_bound`, `cheating_probability_bound`): at most one accepting challenge out of q.

6. **Perfect honest-verifier zero knowledge** (§7): `simulate_accepts`, `zkpop_perfect_hvzk` (explicit bijection of randomness), `real_image_eq_sim_image` (equality of transcript sets).

7. **Non-interactive version** (§8): `fsProve`, `fs_complete`, `fs_special_soundness`.

8. **A concrete instantiation** (§9): a group, a generator, a certificate and a digest for which every hypothesis is discharged by computation (`demo_possesses`, `demo_completeness`, `demo_extraction`).

1.3 What is *not* claimed

We are explicit about the boundary between what is machine-checked and what is assumed; see also §10.

- **Hiding of the digest is a computational assumption.** Perfect honest-verifier zero knowledge — the statement that the *transcript* of a protocol run reveals nothing — is proved unconditionally. That the published digest $y = g^x$ itself hides x is the discrete-logarithm assumption; it is a computational hypothesis about the chosen group and is *not* proved here, nor is any statement of that form claimed.

- **The soundness bound is a counting statement.** We prove that the set of challenges on which a witness-less prover is accepted has cardinality at most 1, out of a challenge space of size q. We do not build a probability space; the phrase "probability at most $1/q$" is the standard reading of that cardinality bound under a uniform challenge.

- **Honest verifier.** Zero knowledge is proved in the honest-verifier sense (the challenge is drawn independently of the commitment). Malicious-verifier ZK and the Fiat–Shamir random-oracle analysis are not formalized; only completeness and special soundness of the non-interactive variant are.

- **Fragment.** The §12 claim language is a fixed fragment (boxedness, emitter output, reference-count balance, interlingua round-tripping, type distinctness, conjunction, reflection), not arbitrary propositions. `check_complete` is completeness for that fragment.

2 Background: the artifact whose possession is proved

2.1 The claim language and its checker

§12 of the development defines an inductive type `Claim` of object-level statements about the micro-core FFI system together with its meaning function `Claim.holds : Claim → Prop`, an inductive type `Cert` of syntactic proof objects, and a total computable checker `check : Cert → Option Claim`. Two theorems of that module are used here:

```
theorem check_sound : forall (p : Cert) (c : Claim), check p = some c -> Claim.holds
    c
theorem certDecode_certEncode (p : Cert) : certDecode (certEncode p) = some p
```

`check_sound` is what makes possession of a certificate meaningful: a certificate that the checker accepts for c witnesses that c is true. `certEncode` is the postfix token stream of a certificate — a list of `CTok` — read back by a two-sorted stack machine; the second theorem says the wire format is lossless.

2.2 Certificates as numbers

A certificate must become a number before it can become an exponent. The token alphabet has 19 symbols; we assign each an index below the base 32 and read the token stream as a little-endian base-32 numeral, with an extra most-significant sentinel digit 1 so that trailing zero-index tokens cannot be lost:

$$\texttt{certDigits}(p) \;=\; \big(\texttt{ctokIdx}\text{ applied to }\texttt{certEncode}(p)\big)\mathbin{+\!\!+}[1], \qquad \texttt{certCode}(p) \;=\; \sum_i \texttt{certDigits}(p)_i \cdot 32^{\,i}.$$

Theorem 2.1 (`digits_certCode`)**.** *`Nat.digits` 32 (`certCode` p) = `certDigits` p.*

Theorem 2.2 (`certOfCode_certCode`: the arithmetization is lossless)**.** *For every certificate p, `certOfCode`(`certCode` p) = `some` p.*

Corollary 2.3 (`certCode_injective`)**.** *`certCode` p = `certCode` p′ ⟹ p = p′.*

The inverse `certOfCode` is a genuine decision procedure: it recomputes the digits, checks the sentinel, maps indices back to tokens, and runs the §12 stack-machine decoder. It rejects junk — for instance `certOfCode` 0 = `none`.

3 Exponentiation by a residue

Let G be a commutative group written multiplicatively and $q > 0$. For $e \in \mathbb{Z}/q\mathbb{Z}$ we write `zpw` y $e = y^{e.\mathrm{val}}$, using the canonical representative $e.\mathrm{val} \in \{0, \dots, q-1\}$. The operation behaves like exponentiation by a residue exactly for elements killed by q, and that is the only property of the group ever used.

Lemma 3.1 (`zpw_add`, `zpw_mul`, `zpw_sub`, `zpw_one`, `zpw_pow_q`)**.** *If $y^q = 1$ then for all $a, b \in \mathbb{Z}/q\mathbb{Z}$:*

$$y^{a+b} = y^a y^b, \qquad y^{ab} = (y^a)^b, \qquad y^{a-b} = y^a (y^b)^{-1}, \qquad y^1 = y \ (q > 1), \qquad (y^a)^q = 1.$$

Lemma 3.2 (`zpw_injective`: uniqueness of the discrete logarithm)**.** *If $\mathrm{ord}(g) = q$ then $e \mapsto g^e$ is injective on $\mathbb{Z}/q\mathbb{Z}$.*

4 The protocol

Public parameters. A commutative group G, a prime q, a generator g with $\mathrm{ord}(g) = q$.

Statement. A claim c and a digest $y \in G$. The assertion is

```
def Possesses (g y : G) (q : Nat) (c : Claim) : Prop :=
  exists p : Cert, check p = some c and certCode p < q and y = zpw g (certCode p :
      ZMod q)
```

i.e. *y is the digest of a certificate that the §12 checker accepts for c.*

Witness. The certificate p_0; equivalently the exponent $x = \texttt{certCode}\ p_0 \bmod q$.

Moves. With $\texttt{Transcript} = (a \in G,\ e \in \mathbb{Z}/q\mathbb{Z},\ z \in \mathbb{Z}/q\mathbb{Z})$:

1. *Commitment.* The prover picks a nonce $k \in \mathbb{Z}/q\mathbb{Z}$ and sends $a = g^k$.
2. *Challenge.* The verifier sends a uniform $e \in \mathbb{Z}/q\mathbb{Z}$.
3. *Response.* The prover sends $z = k + ex$.

The verifier accepts iff

$$\texttt{accepts}\ g\ y\ (a, e, z)\ :\equiv\ g^z = a \cdot y^e.$$

Simulator and extractor.

$$\texttt{simulate}\ g\ y\ e\ z = \big(g^z (y^e)^{-1},\ e,\ z\big), \qquad \texttt{extractExp}\ t_1\ t_2 = \frac{z_1 - z_2}{e_1 - e_2}, \qquad \texttt{extract}\ t_1\ t_2 = \texttt{certOfCode}\big((\texttt{extrac}$$

Theorem 4.1 (`possession_sound`)**.** *`Possesses` $g\ y\ q\ c \implies$ `Claim.holds` c.*

This is the bridge to §12: possession is not a bare assertion, it entails the truth of the claim, by `check_sound`.

5 Completeness

Theorem 5.1 (`completeness`)**.** *Let $g^q = 1$ and $y = g^x$. Then for all nonces k and all challenges e, `accepts` $g\ y$ (`prove` $g\ x\ k\ e$).*

Proof (formalized). $g^{k+ex} = g^k g^{ex} = g^k (g^x)^e = a \cdot y^e$, using `zpw_add`, commutativity of $\mathbb{Z}/q\mathbb{Z}$ and `zpw_mul`. □

An honest prover therefore never fails — there is no completeness error.

6 Knowledge soundness

Theorem 6.1 (`special_soundness`)**.** *Let q be prime, $g^q = 1$, $y^q = 1$, and let $t_1 = (a, e_1, z_1)$ and $t_2 = (a, e_2, z_2)$ be accepting transcripts with the same commitment and $e_1 \neq e_2$. Then $g^{(z_1 - z_2)/(e_1 - e_2)} = y$.*

Proof (formalized). Dividing the two verification equations kills the shared commitment: $g^{z_1 - z_2} = (a y^{e_1})(a y^{e_2})^{-1} = y^{e_1 - e_2}$. Put $d = e_1 - e_2 \neq 0$, invertible because $\mathbb{Z}/q\mathbb{Z}$ is a field. Then $y = y^{dd^{-1}} = (y^d)^{d^{-1}} = (g^{z_1 - z_2})^{d^{-1}} = g^{(z_1 - z_2) d^{-1}}$. □

The point of a *proof of possession* is that the extracted exponent is not merely a number: it is the certificate.

Theorem 6.2 (`zkpop_extracts_certificate`; main theorem)**.** *Let q be prime,* $\mathrm{ord}(g) = q$*, let p_0 be a certificate with* $\mathtt{check}\, p_0 = \mathtt{some}\, c$ *and* $\mathtt{certCode}\, p_0 < q$*, and let $y = g^{\mathtt{certCode}\, p_0}$ be the published digest. If t_1, t_2 are accepting transcripts with the same commitment and distinct challenges, then*

$$\mathtt{extract}\, t_1\, t_2 = \mathtt{some}\, p_0, \qquad \mathtt{Possesses}\, g\, y\, q\, c, \qquad \mathtt{Claim.holds}\, c.$$

Proof (formalized). `special_soundness` gives $g^{\mathtt{extractExp}\, t_1\, t_2} = y = g^{\mathtt{certCode}\, p_0}$; `zpw_injective` (using $\mathrm{ord}(g) = q$) makes the exponents equal; the bound $\mathtt{certCode}\, p_0 < q$ makes the canonical representative of that residue equal to $\mathtt{certCode}\, p_0$ on the nose; `certOfCode_certCode` returns p_0; and `check_sound` gives $\mathtt{Claim.holds}\, c$. □

So a prover that can answer two distinct challenges on one commitment *is* in possession of the artifact, in the strongest operational sense: a verifier standing next to two such transcripts can print the certificate.

Theorem 6.3 (`cheating_challenge_bound`)**.** *Let q be prime, $g^q = 1$, $y^q = 1$, and suppose y has no discrete logarithm base g in $\mathbb{Z}/q\mathbb{Z}$. Then for every prover strategy consisting of a fixed commitment a and an arbitrary response function* $\mathrm{resp} : \mathbb{Z}/q\mathbb{Z} \to \mathbb{Z}/q\mathbb{Z}$*,*

$$\#\{\, e \in \mathbb{Z}/q\mathbb{Z} \;:\; g^{\mathrm{resp}(e)} = a\, y^e \,\} \;\le\; 1.$$

Corollary 6.4 (`cheating_probability_bound`)**.** *Under the same hypotheses, the fraction of the challenge space on which the strategy succeeds is at most $1/q$; equivalently, a uniform challenge makes the cheating probability at most $1/q$, and n independent repetitions q^{-n}.*

7 Zero knowledge

The simulator takes only public data — g, y, the challenge e, and a random z — and never touches the witness.

Theorem 7.1 (`simulate_accepts`)**.** *For all e, z:* $\mathtt{accepts}\, g\, y\, (\mathtt{simulate}\, g\, y\, e\, z)$.

Theorem 7.2 (`zkpop_perfect_hvzk`)**.** *Let $g^q = 1$ and $y = g^x$. For every challenge e the map* $\mathtt{respShift}\, x\, e : k \mapsto k + ex$ *is a bijection of $\mathbb{Z}/q\mathbb{Z}$, and for every nonce k*

$$\mathtt{prove}\, g\, x\, k\, e \;=\; \mathtt{simulate}\, g\, y\, e\, (\mathtt{respShift}\, x\, e\, k).$$

Because `respShift` is an `Equiv`, a uniformly random nonce induces a uniformly random simulator input: the honest and simulated transcript distributions are *equal*, not merely close. The set-level form is proved as well.

Theorem 7.3 (`real_image_eq_sim_image`)**.** $\{\mathtt{prove}\, g\, x\, k\, e : k \in \mathbb{Z}/q\mathbb{Z}\} = \{\mathtt{simulate}\, g\, y\, e\, z : z \in \mathbb{Z}/q\mathbb{Z}\}$ *as finite sets of transcripts.*

Hence the verifier's entire view of an honest run is computable from the public statement alone, and so carries no information about which certificate — indeed, about whether any particular certificate — underlies the digest.

8 The non-interactive proof

Replacing the verifier's coin flips by a public challenge derivation $H : G \to \mathbb{Z}/q\mathbb{Z}$ applied to the commitment turns the interaction into a single publishable string (a, e, z):

$\texttt{fsProve}\ H\ g\ x\ k = \texttt{prove}\ g\ x\ k\ (H(g^k)), \qquad \texttt{fsAccepts}\ H\ g\ y\ t\ :\equiv\ t.e = H(t.a) \wedge \texttt{accepts}\ g\ y\ t.$

Theorem 8.1 (`fs_complete`)**.** *For any H, any x with $y = g^x$ and any nonce k:* `fsAccepts` $H\ g\ y$ (`fsProve` $H\ g\ x$

Theorem 8.2 (`fs_special_soundness`)**.** *Two non-interactively accepted transcripts sharing a commitment but with distinct challenges still yield the discrete logarithm, hence (as above) the certificate.*

As noted in §1.3, the random-oracle argument that makes Fiat–Shamir sound against a single adversary is not formalized; what is formalized is that the transformation preserves completeness and that the extractor still applies.

9 A concrete instantiation

To show that no hypothesis above is vacuous, the module fixes

$q = 37957$ (prime), $\quad G = \big(\mathbb{Z}/q\mathbb{Z}, +\big)$ written multiplicatively, $\quad g = 1, \quad p_0 =$ `boxedR (array nat)`,

so that `check` p_0 = `some` (`isBoxed` (`array nat`)), `certCode` $p_0 = 37952 < q$, and $\mathrm{ord}(g) = q$; the digest is $y = g^{37952}$. The Lean theorems `demo_possesses`, `demo_completeness` and `demo_extraction` instantiate possession, completeness and extraction with every side condition discharged.

Remark 9.1. This particular group has easy discrete logarithms, so it demonstrates the protocol's completeness, soundness and simulator — not the computational hiding of the digest, which requires a DL-hard group. Substituting such a group changes nothing in the proofs: every theorem above is stated for an arbitrary commutative group with an element of prime order q.

10 Discussion

10.1 What the paper's title claim amounts to

Composing the results: if the prover publishes y and can answer two distinct challenges on one commitment, then *a certificate exists, the extractor names it, the checker accepts it, and the claim it certifies is true* — while each individual honest interaction is reproducible by a witness-free simulator. The three properties one wants of a proof of possession therefore all hold, and all are machine-checked:

Property	**Lean theorem**	**Strength**
Completeness	`completeness`	perfect (no error)
Knowledge soundness	`zkpop_extracts_certificate`	extractor outputs p_0
Soundness error	`cheating_probability_bound`	$\le 1/q$ per round
Zero knowledge	`zkpop_perfect_hvzk`	perfect, honest verifier
Losslessness of encoding	`certOfCode_certCode`	exact round trip
Truth of the claim	`possession_sound` + `check_sound`	from §12

10.2 Relation to the rest of the development

The digest is taken of the *wire form* of a certificate, the same object that §12 ships across the FFI boundary and that §11's interlingua round-trips. A foreign runtime can therefore play either role: given the certificate it re-checks it (`foreign_check`), and given only the digest and a transcript it verifies possession. The reflection layer of §12 composes with this: since "w is the wire form of a certificate the checker accepts for c" is itself a claim with a certificate (`reflect_complete`), one can prove possession *of a proof-of-a-proof* by exactly the same protocol, applied to the reflected certificate.

10.3 Limitations and future work

Beyond the boundaries listed in §1.3: the protocol proves possession of *some* certificate with the published digest, so the statement is only as informative as the digest is bound to the claim — a verifier must obtain y and c from the same trusted publication. A natural extension is to prove possession of a certificate for a claim *chosen by the verifier* after the digest is fixed, which requires proving a relation between the committed exponent and the checker's output inside the proof system, i.e. arithmetizing `check` itself. Formalizing the probability space, malicious-verifier ZK, and the random-oracle analysis of the Fiat–Shamir transform are the other obvious next steps.

Reproducing

```
lake build -- builds the whole project, including the ZKPoP module
```

The module is `RequestProject/MicroLeanZKPoP.lean` (namespace `MicroLean.ZKPoP`). It contains no `sorry`; the theorems cited above depend only on `propext`, `Classical.choice` and `Quot.sound`, which can be confirmed with `#print axioms`.